



PRŮVODCE DO KAPSY

Ročníková práce

Autor	Nella Pražáková
Obor	Informační technologie
Vedoucí práce	Ing. Tomáš Kazda, DiS.
Školní rok	2025/2026
Počet stran	27
Počet slov	4221

Anotace

Práce se zabývá vývojem webové aplikace pro usnadnění orientace návštěvníků při pohybu v areálech školy během akcí pro veřejnost. Projekt vychází z potřeby nahradit zastaralé statické papírové mapy moderním řešením, které je jednoduše dostupné z mobilního zařízení.

Aplikace poskytuje uživateli přehledný seznam otevřených učeben a zájmových stanovišť v budově školy. Součástí aplikace je také administrační rozhraní, které slouží ke snadné správě a aktualizaci bodů na mapě.

Summary

This work focuses on the development of a web application designed to facilitate visitor orientation within the school premises during public events, such as open house days. The project is based on the need to replace outdated static paper maps with a modern digital solution easily accessible from mobile devices.

The application provides users with a clear list of open classrooms and points of interest within the school building. It also features an administrative interface used for easy management and updating of these points without the need for code modifications.

Čestné prohlášení

Prohlašuji, že jsem tuto práci vypracoval/a samostatně. V práci jsem použil/a pouze zdroje uvedené v seznamu literatury a všechny převzaté části, včetně textů, obrázků, kódu a výstupů umělé inteligence, jsou řádně označeny a citovány.

V Liberci dne 05.06.2026

.....
Nella Pražáková

Obsah

Úvod.....	1
1 Analýza řešení	2
1.1 Popis problému.....	2
1.2 Katalog požadavků.....	2
1.2.1 Funkcionální požadavky.....	2
1.2.2 Nefunkcionální požadavky	3
1.3 Volba technologií.....	3
2 Návrh aplikace.....	5
2.1 Návrh uživatelského rozhraní.....	5
2.1.1 Analýza uživatelů	6
2.1.2 Uživatelský scénář	6
2.2 Návrh administračního rozhraní	6
2.3 Návrh studentského rozhraní	7
2.4 Struktura databáze	8
2.5 Návrh API endpointů	9
3 Realizace	10
3.1 Realizace klientské části aplikace	10
3.1.1 Testování uživatelského rozhraní	11
3.2 Realizace administrační a studentské části	12
3.2.1 Testování administračního rozhraní.....	13
3.3 Realizace databáze a API	14
3.4 Nasazení aplikace.....	15
4 Technické výzvy a jejich řešení.....	16
4.1 Responzivní práce s SVG mapou	16
4.2 Převod SVG mapy do datové struktury	16
4.3 Migrace databáze během vývoje	16
4.4 Ruční opravy mapových podkladů.....	16
4.5 Synchronizace dat mezi klientskou a administrační částí.....	17
Závěr.....	18
Seznam zkratek a odborných výrazů.....	19
Seznam obrázků.....	20
Použité zdroje	21

A.	Seznam přiložených souborů	I
----	----------------------------------	---

Úvod

Tato práce se zabývá návrhem a vývojem webové aplikace pro usnadnění orientace v areálech školy během akcí pro veřejnost. Aplikace využívá interaktivní mapu, ve které jsou zobrazena stanoviště, místnosti a další informace vztahující se k aktuálně pořádané akci.

Součástí práce je také administrační rozhraní pro správu obsahu mapy.

Dokument popisuje návrh a postup vývoje, použitou databázovou strukturu, realizaci uživatelského i administračního rozhraní a řešení technických překážek vzniklých během vývoje.

Při vývoji aplikace a tvorbě dokumentace byly jako pomocné nástroje využity systémy umělé inteligence ChatGPT a Claude. Konkrétní ukázky použitých promptů jsou uvedeny v příloze práce.

1 Analýza řešení

Cílem této kapitoly je podrobný rozbor navrhovaného prostředí. Analýza slouží jako nezbytný podklad pro následující návrh a samotnou implementaci.

1.1 Popis problému

Orientace v areálech školy může být pro návštěvníky během akcí nepřehledná. V současnosti je tento problém řešen plánsky s nákresem budov, pater a jednotlivých učeben.

Papírové plánsky jsou jednoduché na použití, avšak méně praktické, protože neumožňují interaktivní práci s obsahem a jejich aktualizace je časově náročná. Při změně stanovišť je nutné mapu znovu upravit a vytisknout. Kvůli omezenému prostoru navíc neobsahují doplňující informace o programu, učebnách nebo oborech.

Mapové aplikace, například Google Mapy nebo Mapy.com, jsou sice interaktivní a dostupné pro většinu návštěvníků, ale kvůli nedostatečné podrobnosti budov je lze využít především pro cestu k areálu, nikoli pro orientaci uvnitř školy.

Z těchto důvodů bylo rozhodnuto vytvořit specializované řešení zaměřené na orientaci návštěvníků uvnitř školního areálu. Navrhovaná aplikace má tento problém zjednodušit pomocí interaktivní mapy dostupné z mobilního zařízení. Cílem není vytvořit plnohodnotný navigační systém, ale přehledný nástroj, který návštěvníkům zobrazí stanoviště, místnosti a související informace a usnadní jim pohyb během akce.

1.2 Katalog požadavků

Tato kapitola definuje klíčové vlastnosti budoucí aplikace. Požadavky jsou rozděleny do dvou hlavních částí: **funkcionální**, které specifikují konkrétní funkce a **nefunkcionální**, které určují vlastnosti systému, například responzivitu.

Pro určení důležitosti jednotlivých bodů je dále použita stupnice priorit:

- **Vysoká (Must have):** funkce nezbytné pro základní provoz aplikace
- **Střední (Should have):** užitečné funkce pro komfort uživatele
- **Nízká (Could have):** doplňkové funkce, v případě časové rezervy

1.2.1 Funkcionální požadavky

ID	Název	Popis	Priorita
F01	Zobrazení mapy	Systém zobrazí interaktivní plán školy s vyznačenými stanovišti.	Vysoká
F02	Detail stanoviště	Po interakci s bodem na mapě se zobrazí podrobné informace o stanovišti.	Vysoká

F03	Přepínání pater	Možnost přepínat mezi patry budovy.	Vysoká
F04	Filtrování stanovišť	Uživatel může filtrovat stanoviště podle typu a specializace.	Střední
F05	Autentizace	Přihlášení probíhá pomocí ASP.NET Identity a předem definovaných hesel.	Vysoká
F06	Role a oprávnění	Systém rozlišuje přístup do administračního rozhraní na dvě úrovně.	Vysoká
F07	Omezené studentské rozhraní	Uživatel s rolí studenta vidí pouze stránky určené na doplnění obsahu.	Nízká
F08	Administrační rozhraní	Správce má CRUD přístup ke správě bodů, akcí a dalších dat.	Vysoká
F09	Zobrazení aktivní akce	Systém načítá stanoviště a informace z aktuálně aktivní akce.	Vysoká

Tabulka 1: Funkcionální požadavky

1.2.2 Nefunkcionální požadavky

ID	Název	Popis	Priorita
NF01	Mobile-First Design	Uživatelské rozhraní je optimalizováno pro mobilní telefony.	Vysoká
NF02	Responzivita	Aplikace je přizpůsobena různým velikostem displejů.	Vysoká
NF03	Rychlost odezvy	Přijatelný čas načtení úvodní mapy díky optimalizaci SVG a dalších prostředků.	Střední
NF04	Intuitivní ovládání	Srozumitelné rozhraní pro všechny věkové skupiny návštěvníků.	Vysoká
NF05	Výkon (Lighthouse)	Aplikace dosahuje skóre Lighthouse Performance alespoň 80 bodů.	Střední
NF06	Bezpečnost	Administrační rozhraní je chráněno autentizací uživatele.	Vysoká
NF07	Konzistence UI	Aplikace používá jednotné barvy, ikony a vizuální styl.	Střední

Tabulka 2: Nefunkcionální požadavky

1.3 Volba technologií

Pro realizaci aplikace byly zvoleny technologie umožňující vývoj responzivního uživatelského rozhraní, administrační části, serverové logiky a databázového úložiště.

Oblast	Technologie	Důvod použití
Frontend	React 19	Komponentová architektura umožňuje znovupoužití UI prvků, předchozí znalost a zkušenosti
Build tool	Vite [9]	HMR umožňoval okamžité zobrazení změn, nativní podpora TypeScriptu
Jazyk	Typescript	Typová bezpečnost, odhalení chyb při práci s daty z API již při kompilaci, ne až při běhu aplikace
Backend	ASP.NET Core	Sdílení jednoho backendu (REST API i Razor Pages) snížilo duplicitu kódu
ORM	Entity Framework Core [8]	Možnost psát databázový model přímo jako C# třídy místo SQL dotazů
Databáze	SQLite	Databáze uložena v jediném souboru umožňuje lehké zálohování, při nasazení nebyla potřeba instalace serveru
Návrh UI	Figma	Vestavěné prototypování umožnilo simulovat přechody mezi obrazovkami před zahájením implementace
Práce s mapou	Knihovna <i>react-zoom-pan-pinch</i> [13]	Podpora dotykového ovládání bez nutnosti vlastní implementace event listenerů
Autentizace	ASP.NET Identity	Správa uživatelů a rolí, oddělení přístupu na úrovni jednotlivých Razor Pages

Tabulka 3: Přehled volby technologií

2 Návrh aplikace

V této části práce jsou popsány postupy návrhu jednotlivých částí aplikace. Pro tvorbu vizuálních návrhů byl zvolen online nástroj Figma, a to díky jednoduchému ovládání a možnosti vytvářet komplexní návrhové materiály.

2.1 Návrh uživatelského rozhraní

Uživatelské rozhraní bylo navrženo jako jednostránková aplikace s důrazem na mobilní zařízení [11]. Hlavní obrazovka je navržena jako jedna dominantní plocha s mapou, aby uživatel nebyl nucen přepínat mezi více stránkami. Ovládací prvky jsou umístěny do postranního panelu, který nezasahuje do mapy.

Návrh vycházel z předpokladu, že hlavní cílovou skupinou jsou návštěvníci školních akcí, kteří aplikaci používají na mobilních telefonech. Z tohoto důvodu byl zvolen jednoduchý layout s důrazem na okamžité zobrazení mapy bez nutnosti navigace v menu. Uživatel nemá být přehlcen informacemi, chce se pouze dozvědět, kde najde jaké stanoviště.

Vizuální návrh byl vytvořen s důrazem na jednoduchost a okamžitou orientaci. Cílem bylo minimalizovat množství ovládacích prvků a ponechat pouze informace nezbytné pro orientaci v budově pro široké spektrum uživatelů.

Hlavní pozornost uživatele je soustředěna na mapu jako na primární funkci, viz obrázek níže. Ostatní informace (filtry, detail bodu) jsou sekundární a zobrazují se pouze na vyžádání uživatele. Barevné schéma bylo zvoleno s důrazem na dostatečný kontrast a čitelnost v souladu s doporučeními WCAG [12]. Je také doplněno o pár barev převzatých ze školního webu, to pomáhá vytvořit vizuální návaznost mezi existujícími školními materiály a novou aplikací. Typografie [10] byla zvolena s ohledem na čitelnost na mobilních zařízeních. Vizuální inspirace pro rozvržení byla hledána na platformě Pinterest s důrazem na mobilní aplikace podobného charakteru.



Obrázek 1: Ukázka návrhu uživatelského rozhraní v programu Figma

2.1.1 Analýza uživatelů

Hlavní cílovou skupinou aplikace jsou návštěvníci dnů otevřených dveří a dalších akcí pro veřejnost. Jedná se zejména o uchazeče o studium a jejich rodiče, kteří budovu školy navštěvují poprvé a neznají její uspořádání.

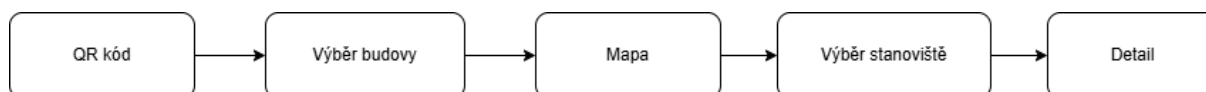
Persona	Charakteristika	Problém	Potřeba
Uchazeč o studium	Student, 14–16 let, navštěvuje školu poprvé	Budovy školy nezná a neví, kde hledat stanoviště	Rychle zjistit, kde se nacházejí místa, která ho zajímají
Doprovod uchazeče	Doprovází uchazeče, není technicky zaměřený	Neorientuje se v budově; aplikaci používá pouze jednorázově	Jednoduché a jasné rozhraní, bez složitého ovládání
Organizátor	Spravuje obsah akce	Nedostatek času pro dlouhé přepisování informací	Aktualizovat stanoviště bez zásahu do kódu

Tabulka 4: Analýza uživatelů aplikace

Sekundární skupinou jsou organizátoři akcí, kteří potřebují rychle aktualizovat informace o stanovištích.

2.1.2 Uživatelský scénář

Na základě analýzy uživatelů byl vytvořen jednoduchý uživatelský scénář:



Obrázek 2: Schéma toku uživatele

Návštěvník po příchodu do budovy načte aplikaci pomocí QR kódu umístěného u vstupu. Po otevření aplikace si na úvodní obrazovce vybere areál k zobrazení. Následně okamžitě vidí mapu aktuálního podlaží bez nutnosti registrace nebo další navigace. Pokud hledá konkrétní obor, využije filtr specializací. Po kliknutí na stanoviště se zobrazí detailní informace o učebně. Celý proces je navržen tak, aby uživatel dosáhl požadované informace s minimálním počtem kroků.

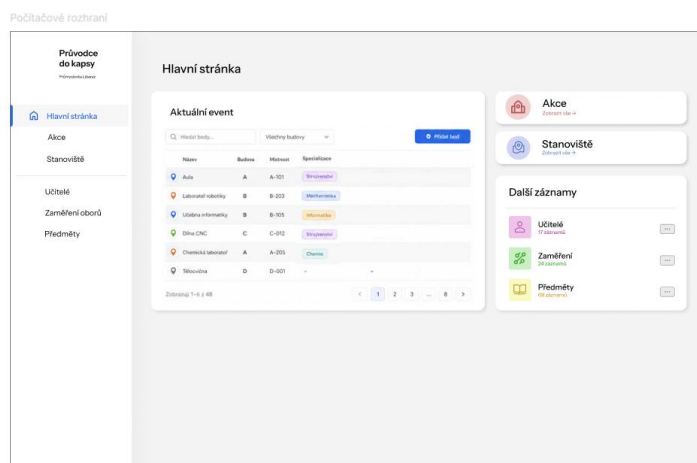
2.2 Návrh administračního rozhraní

V rámci návrhu administračního rozhraní bylo vytvořeno několik obrazovek. Kromě hlavní obrazovky mezi ně patří stránky pro vytváření, editaci a mazání jednotlivých položek v databázi. Všechny stránky jsou navrženy pro desktopové zobrazení. Součástí návrhu administrační části je také systém autentizace uživatelů.

Barevné schéma vychází z vizuální identity školy. Díky tomu aplikace působí jako přirozená součást školního informačního systému a nikoliv jako samostatný nesouvisející

produkt. Vizuál je minimalistický, bylo ponecháno pouze nezbytné minimum prvků pro usnadnění práce s rozhraním. Jsou použity jasné názvy, ikony a vyhledávací pole pro zpříjemnění zadávání informací.

Návrh rozhraní byl během vývoje průběžně upravován na základě zpětné vazby získané při testování. Některé prvky navržené ve Figmě byly během implementace zjednodušeny nebo přesunuty tak, aby lépe odpovídaly skutečnému způsobu používání aplikace.

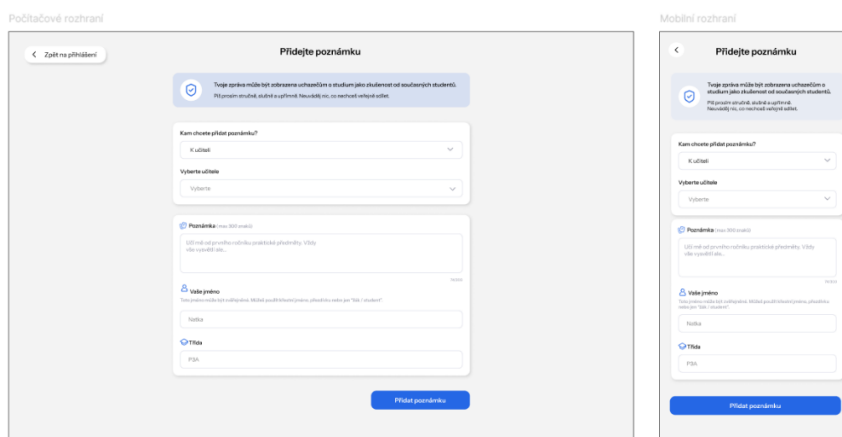


Obrázek 3: Ukázka administračního rozhraní v programu Figma

2.3 Návrh studentského rozhraní

Studentské rozhraní bylo navrženo jako zjednodušená část administrace a využívá stejného systému přihlášení. Rozhraní slouží především k doplňování poznámek k jednotlivým předmětům a učitelům, které pomáhají obohatit informační texty.

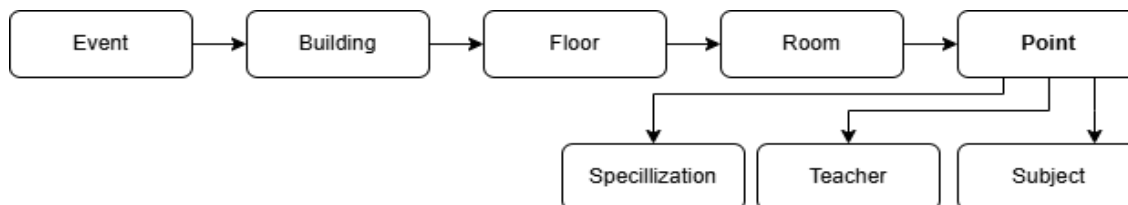
Vzhled se také drží minimalismu. Barevná kombinace tvoří neutrální tóny s jednou primární barvou, převzatou z loga školy.



Obrázek 4: Ukázka návrhu studentského rozhraní v programu Figma

2.4 Struktura databáze

Data aplikace jsou uložena v databázi SQLite. Model se skládá z několika tabulek, které jsou navzájem propojeny pomocí relace. Použitá databázová struktura byla navržena s ohledem na jednoduché rozšiřování aplikace o další budovy a akce bez nutnosti zásadních změn v datovém modelu.



Obrázek 5: Schéma databázové struktury

Databázový model obsahuje entity: Event (akce), Building (budova), Floor (podlaží), Room (místnost), Point (stanoviště), Teacher (učitel), Subject (předmět) a Specialization (specializace). Každá akce obsahuje více stanovišť, která jsou vázána na konkrétní budovu a patro.

Mezi entitami jsou definovány relace různých typů. Hierarchie prostoru je řešena řetězcem relací 1:N: jedna budova (Building) obsahuje více podlaží (Floor), každé podlaží obsahuje více místností (Room). Stanoviště (Point) je vázáno na právě jednu místnost.

Vazba mezi akcí a stanovištěm je realizována spojovací tabulkou EventPoint, která umožňuje přiřadit jedno stanoviště více akcím a jedna akce může obsahovat více stanovišť, jde tedy o relaci M:N.

```
public class EventPoint
{
    public int EventId { get; set; }
    public Event? Event { get; set; }

    public string PointId { get; set; } = string.Empty;
    public Point? Point { get; set; }
}
```

Stanoviště má přiřazenu právě jednu specializaci (Specialization) a volitelně jeden předmět (Subject). Vazba Point a Teacher je realizována spojovací tabulkou, protože jedno stanoviště může mít více učitelů, tedy relace M:N.

2.5 Návrh API endpointů

API bylo navrženo jako centrální vrstva mezi databází a jednotlivými částmi systému. Klientská aplikace využívá endpoint `/api/referencedata/points` pro načtení stanovišť aktuálně aktivní akce. Administrační rozhraní využívá stejné datové zdroje při vytváření a úpravách obsahu. Díky tomuto přístupu jsou všechna data spravována na jednom místě a změny provedené v administraci se okamžitě projeví v klientské části aplikace bez nutnosti úprav frontendového kódu.

Metoda	Endpoint	Popis
GET	<code>/api/referencedata/events</code>	Vrátí aktuálně viditelné akce
GET	<code>/api/referencedata/points</code>	Vrátí body aktivní akce
GET	<code>/api/referencedata/teachers</code>	Vrátí seznam učitelů
GET	<code>/api/referencedata/subject</code>	Vrátí seznam předmětů

Tabulka 5: Přehled vybraných endpointů

Endpoint `/api/referencedata/events` slouží k načtení seznamu aktuálně dostupných akcí. Endpoint `/api/referencedata/points` vrací pouze stanoviště přiřazená aktivní akci, což umožňuje měnit obsah aplikace bez zásahu do mapových podkladů.

3 Realizace

3.1 Realizace klientské části aplikace

Klientská část aplikace byla implementována pomocí knihovny React 19 s využitím jazyka TypeScript.

Pro zobrazení mapy byl zvolen vlastní SVG podklad pro snadnou práci s body. Pomocí pomocného skriptu byl SVG podklad převeden do JSON souboru.

Načítání statických mapových podkladů je realizováno pomocí hooku `useEffect()`, který se vykoná po prvním vykreslení komponenty. Nejprve je načten soubor `map.json` obsahující seznam budov a následně jsou asynchronně načítána data jednotlivých podlaží. Výsledná data jsou uložena do stavu aplikace pomocí funkce `setBuildings()`.

```
useEffect(()                                     =>                                     {
  async function loadData()                       {
    const mapData = await fetchJson<{ buildings: BuildingData[]
  }>('/data/map.json');
    setBuildings(buildingsWithRooms);
  }

  loadData();
}, []);
```

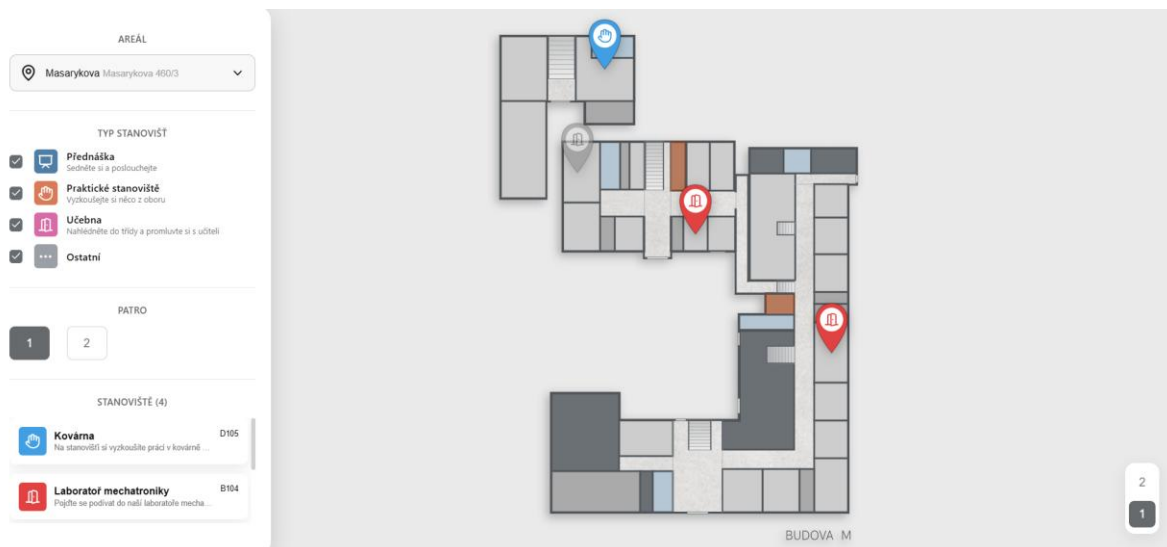
Po načtení aplikace dojde k získání dat aktuálně aktivní akce z API. Na základě těchto dat jsou následně vykresleny jednotlivé body v mapě a zpřístupněny související informace. Díky tomuto přístupu není potřeba měnit klientskou část aplikace při každé změně obsahu.

Součástí implementace bylo také vytvoření systému filtrů, který umožňuje zobrazit pouze vybrané typy stanovišť. Uživatel si může současně přepínat jednotlivá patra budovy a zobrazovat detailní informace o konkrétním bodu.

```
const toggleType = (type: PointIcon) => {
  onChange(previous => {
    if (previous.includes(type)) {
      return previous.filter(item => item !== type);
    }

    return [...previous, type];
  });
};
```

Filtrování stanovišť je realizováno pomocí seznamu aktivních typů. Po kliknutí na filtr je vybraný typ přidán do seznamu aktivních filtrů nebo z něj odebrán. Díky tomuto přístupu je možné současně filtrovat více kategorií stanovišť bez nutnosti opakovaného načítání dat z backendu.



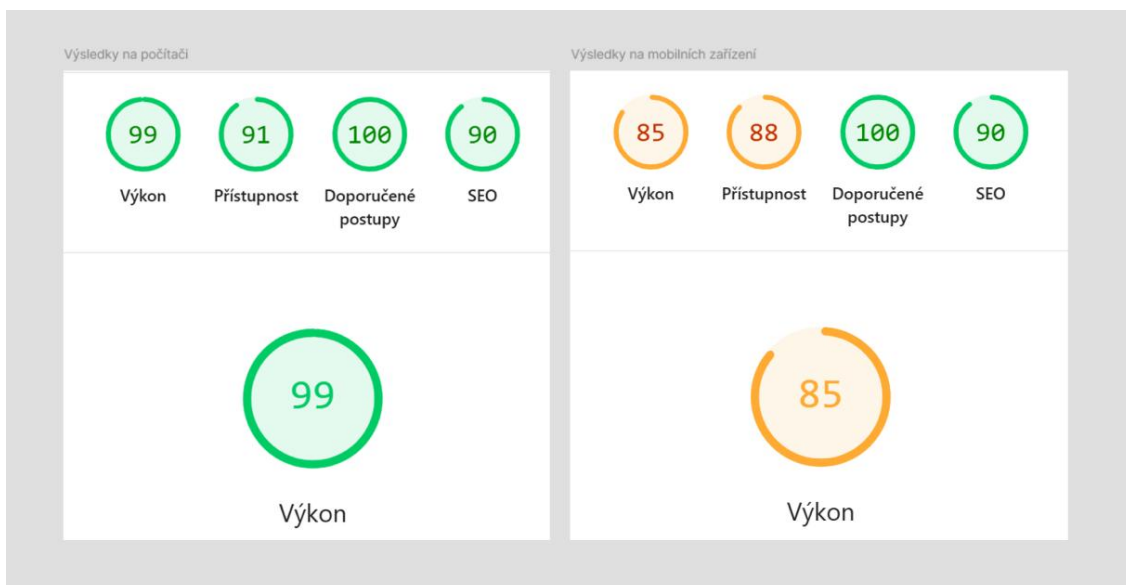
Obrázek 6: Implementovaná mapa s aktivními body

3.1.1 Testování uživatelského rozhraní

Uživatelské rozhraní bylo testováno 15 uživateli. Do testování byli zapojeni studenti školy, kteří budovu dobře znají, dále mladí lidé z jiných škol, kteří školu nikdy nenavštívili, a také dospělí návštěvníci. Testování probíhalo na zařízeních s různými velikostmi displejů a operačními systémy. Podle předem definovaných kritérií byla ověřována zejména responzivita, čitelnost mapy a intuitivnost ovládání.

Z 15 testovaných uživatelů dokázali všichni bez pomoci najít požadované stanoviště, přepnout patro a upravit si filtry podle svých preferencí. Nejčastější připomínkou byla absence druhé budovy a nejednoznačné umístění některých odkazů. Na základě této zpětné vazby byly upraveny navigační prvky, vycentrovány texty a doplněny vysvětlující texty.

Nefunkcionální požadavek NF05 (výkon aplikace) byl ověřen pomocí Google Lighthouse, kde aplikace dosáhla průměrného skóre na různých velikostech displejů 92 bodů.



Obrázek 7: Výsledky Google Lighthouse

3.2 Realizace administrační a studentské části

Administrační část aplikace byla implementována pomocí technologie Razor Pages v prostředí ASP.NET Core [6]. Aplikace využívá ASP.NET Identity pro správu uživatelských rolí, přičemž samotné přihlášení je zjednodušeno na systém předem definovaných přístupových hesel, která určují přiřazenou roli.

Aplikace využívá dvě uživatelské role:

- **Admin**, který má přístup k plné správě dat.
- **Student**, který má přístup pouze na stránku určenou k vytváření a úpravě poznámek, které slouží jako doplňující informace určené uchazečům.

```
options.AddPolicy("AdminPolicy", policy =>
    policy.RequireRole("Admin"));

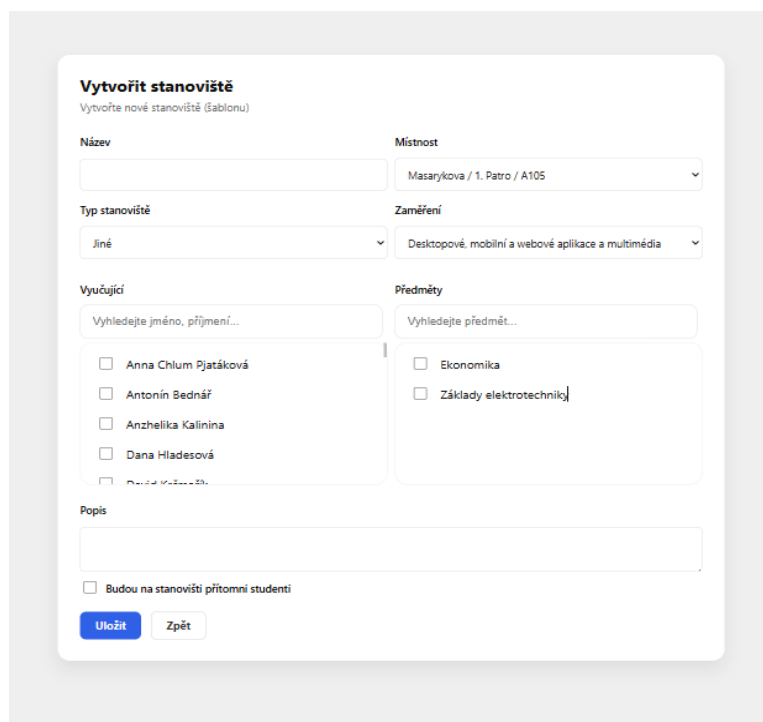
options.AddPolicy("StudentPolicy", policy =>
    policy.RequireRole("Student", "Admin"));
```

Obě role využívají společný přihlašovací formulář. Přihlášení je realizováno pomocí předem definovaných hesel, na jejichž základě je uživateli přiřazena odpovídající role. Pro správu autentizace a autorizace je využita technologie ASP.NET Identity. Po úspěšném přihlášení jsou uživateli zpřístupněny pouze stránky odpovídající jeho roli. Oproti návrhu bylo nutné upravit implementaci rolí tak, aby odpovídala reálnému chování aplikace při přihlašování pomocí předdefinovaných hesel.

Zvolený přístup pomocí předem definovaných sdílených hesel představuje zjednodušení oproti standardní správě individuálních uživatelských účtů. Toto řešení bylo přijatelné vzhledem k omezenému počtu uživatelů administrace, internímu

charakteru nasazení a skutečnosti, že aplikace nepracuje s citlivými nebo osobními údaji. Není proto nezbytné evidovat, který konkrétní uživatel provedl jednotlivé změny. Pro nasazení v produkčním prostředí s větším počtem uživatelů by však bylo vhodné nahradit sdílená hesla individuálními účty s možností správy oprávnění a auditního záznamu změn.

Administrační část byla implementována pomocí Razor Pages jako server-renderované rozhraní nad společným backendem. Toto řešení bylo zvoleno z důvodu rychlejší implementace administračních operací a jednodušší práce s autentizací. Klientská část aplikace (React SPA) a administrační část (Razor Pages) sdílejí společný backend a databázi.



Obrázek 8: Formulář pro vytváření stanovišť

3.2.1 Testování administračního rozhraní

Testování administračního rozhraní bylo zaměřeno zejména na správné fungování CRUD operací, autentizaci uživatelů a omezení přístupu podle uživatelských rolí. Testování provádělo šest uživatelů. Byly odhaleny problémy s editací některých záznamů, které byly způsobeny nesprávným předáváním dat ve formulářích. Na základě získané zpětné vazby byly upraveny některé formuláře pro editaci dat, opraveny chyby při ukládání změn a vylepšena responzivita vybraných stránek. Současně byly upraveny některé texty a popisy pro lepší srozumitelnost rozhraní. Po opravě byly všechny CRUD operace znovu ověřeny.

3.3 Realizace databáze a API

Pro ukládání dat byla zvolena databáze SQLite. Přístup k databázi je realizován pomocí technologie Entity Framework Core [8], která umožňuje práci s daty prostřednictvím objektových modelů aplikace. Databázová vrstva je tvořena třídou DbContext a jednotlivými entitami reprezentujícími ukládaná data.

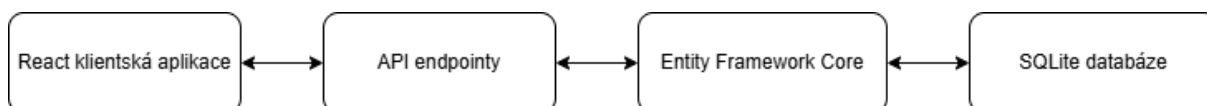
Komunikace mezi klientskou částí aplikace, administrací a databází probíhá prostřednictvím REST API vytvořeného v prostředí ASP.NET Core. Klientská aplikace tak pracuje pouze s daty získanými z API a nemá přímý přístup k databázi. Stejným způsobem jsou data využívána také v administrační části systému. API zároveň slouží jako společný zdroj dat pro React klienta i administrační rozhraní. Při změně dat v administraci není nutné provádět žádné změny v klientské aplikaci.

Důležitou součástí implementace je logika aktivní akce. Ta zajišťuje, že uživatelům jsou zobrazována pouze data vztahující se k aktuálně probíhající události. Výběr aktivních akcí je realizován metodou `GetCurrentlyVisibleEventsAsync()`, která pracuje s příznakem `IsActive`. Metoda nejprve načte všechny akce s příznakem `IsActive` a následně ověří, zda aktuální čas spadá do jejich časového rozsahu.

```
var activeEvents = await _context.Events
.Include(e => e.EventPoints)
.ThenInclude(ep => ep.Point)
.Where(e => e.IsActive)
.AsNoTracking()
.ToListAsync();

var withinDateRange = (!evt.StartDate.HasValue ||
evt.StartDate <= now) && (!evt.EndDate.HasValue ||
evt.EndDate >= now);
```

V průběhu vývoje bylo vytvořeno několik databázových migrací. Ty sloužily zejména k úpravám relačních vazeb mezi entitami, rozšiřování datových modelů a přizpůsobování databázové struktury nově vznikajícím požadavkům aplikace.



Obrázek 9: Schéma toku dat v aplikaci

3.4 Nasazení aplikace

Aplikace byla nasazena do školního sandbox prostředí dostupného pod doménou `sandbox.pslib.cloud`. Nasazení je automatizováno pomocí GitHub Actions. Po každém odeslání změn (push) do hlavní větve je spuštěn workflow, který sestaví aplikaci a nasadí ji na server. Frontend je při lokálním vývoji spouštěn příkazem `npm run dev`, backend příkazem `dotnet run`. V produkčním prostředí jsou obě části sestaveny do finální podoby a provozovány společně pod přidělenou subdoménou. Toto řešení zajišťuje, že testovací prostředí vždy odráží aktuální stav hlavní větve repozitáře.

4 Technické výzvy a jejich řešení

Během realizace aplikace bylo potřeba řešit několik problémů. Některé výzvy vyplynuly až v průběhu vývoje, kdy bylo nutné přizpůsobit původní návrh novým požadavkům nebo technickým omezením. Následující podkapitoly popisují nejvýznamnější problémy a způsob jejich řešení.

4.1 Responzivní práce s SVG mapou

Jedním z problémů bylo zachování správného zobrazení SVG mapy na různých velikostech displejů. Při změně velikosti obrazovky docházelo k odlišnému měřítku mapy a zhoršení ovládání. Řešení využívá dynamické škálování pomocí knihovny `react-zoom-pan-pinch` [13], která umožňuje plynulé přibližování a posouvání mapy. Díky tomu zůstává mapa použitelná na mobilních telefonech i větších obrazovkách.

4.2 Převod SVG mapy do datové struktury

Podklady budov byly vytvořeny ve formátu SVG. Pro efektivní práci v aplikaci bylo potřeba získat informace o jednotlivých místnostech a převést je do strukturované podoby. Z tohoto důvodu byl vytvořen pomocný skript, který z SVG souboru automaticky vygeneroval JSON strukturu obsahující potřebná data o místnostech. Přestože se tím výrazně zjednodušilo zpracování mapových podkladů, u některých místností došlo k nesprávnému převodu souřadnic, a proto bylo nutné jejich data následně upravit ručně.

4.3 Migrace databáze během vývoje

V průběhu vývoje docházelo k opakovaným změnám datového modelu aplikace. Bylo proto nutné vytvářet databázové migrace pomocí `Entity Framework Core` a průběžně upravovat existující databázovou strukturu. Některé změny způsobovaly konflikty mezi staršími migracemi a aktuálním stavem modelů, což vedlo k chybám při vytváření databáze nebo aktualizaci schématu. Řešení spočívalo v úpravě migračních souborů, kontrole relačních vazeb a opakovaném testování migrací na čisté databázi. Díky tomu bylo možné zachovat konzistentní databázovou strukturu i při postupném rozšiřování funkcionality aplikace.

4.4 Ruční opravy mapových podkladů

Přestože byl vytvořen skript pro převod SVG map do datové struktury JSON, některé místnosti byly po převodu vykresleny na nesprávných souřadnicích nebo měly neúplnou geometrii. Důvodem byly rozdíly ve struktuře původních SVG objektů. Bylo proto nutné

ručně kontrolovat výsledná data a opravovat souřadnice problematických místností. Tento proces byl časově náročný, ale umožnil zachovat přesnost mapových podkladů bez nutnosti kompletní ruční digitalizace celé budovy.

4.5 Synchronizace dat mezi klientskou a administrační částí

Aplikace obsahuje dvě rozdílné části: klientskou a administrační. Obě části však pracují se stejnými daty. Výzvou bylo navrhnout architekturu tak, aby nedocházelo k duplicitě logiky ani datových modelů. Řešením bylo vytvoření společného REST API, které slouží jako jediný zdroj dat pro klientskou i administrační část. Díky tomu se změny provedené v administraci okamžitě projeví v uživatelském rozhraní bez nutnosti dalších úprav.

Závěr

Hlavním úkolem bylo navrhnout a vytvořit webovou aplikaci, která usnadní orientaci návštěvníků během akcí pro veřejnost. Aplikace splnila všechny funkční požadavky definované v kapitole 1.2. Bylo implementováno zobrazení interaktivní mapy, detail stanovišť, přepínání pater, filtrování stanovišť, systém autentizace a uživatelských rolí, administrační rozhraní i zjednodušené studentské rozhraní. Současně byly splněny také hlavní nefunkcionální požadavky, zejména responzivita, optimalizace pro mobilní zařízení a bezpečnost přístupu do administrace. Požadavek NF05 na výkon aplikace byl ověřen pomocí nástroje Google Lighthouse, kde aplikace dosáhla průměrného skóre 92 bodů.

Z důvodu časových možností je aplikace aktuálně připravena pro použití s jednou kompletně zpracovanou budovou, která slouží pro testování a ověření všech funkcionalit. Druhá část areálu je v současnosti ve fázi dokončování a bude do systému postupně doplněna. Ruční tvorba mapových podkladů byla časově náročnější, než jsem původně předpokládala. Do budoucna by bylo vhodné tento proces optimalizovat nebo automatizovat. Současně by bylo možné podpořit využívání aplikace například informačními plakáty s QR kódy umístěnými při vstupu a v celém areálu školy.

Náročnou částí projektu byl také vývoj serverové části aplikace. Během práce bylo několikrát nutné upravit návrh databáze a přizpůsobit mu navazující části systému. Užitečnou zkušeností se stala i tvorba administračního rozhraní v technologii Razor Pages.

Jako největší přínos projektu hodnotím získání zkušeností s návrhem kompletní webové aplikace zahrnující frontend, backend, databázi i nasazení. Současně jsem si ověřila význam důkladné analýzy a návrhu datového modelu ještě před zahájením implementace. Některé změny databázové struktury bylo nutné provádět až během vývoje, což vedlo k dodatečným úpravám navazujících částí systému. V budoucích projektech bych proto větší pozornost věnovala návrhu architektury a datových vztahů v počátečních fázích vývoje.

V případě zapojení do běžného používání může aplikace představovat moderní alternativu k tradičním tištěným plánkům a usnadnit tak orientaci návštěvníkům.

Seznam zkratek a odborných výrazů

ORM

Object-Relational Mapping – vrstva mezi databází a kódem

API

Application Programming Interface – rozhraní pro komunikaci mezi systémy

SPA

Single Page Application – jednostránková webová aplikace

CRUD

Create, Read, Update, Delete – operace nad daty

REST

Architektonický styl pro návrh API

HMR

Hot Module Replacement – mechanismus pro aktualizaci kódu bez nutnosti kompletního reloadu stránky

QR

Quick Response – typ dvourozměrného kódu

UI

User Interface – uživatelské rozhraní

UX

User Experience – uživatelská zkušenost

SVG

Scalable Vector Graphics – vektorový grafický formát

JSON

JavaScript Object Notation – textový formát pro ukládání a přenos dat

SQL

Structured Query Language – jazyk pro práci s relačními databázemi

Lighthouse

Nástroj společnosti Google pro měření kvality webových aplikací

Github Actions

Služba pro automatizaci buildů, testování a nasazení aplikací

Seznam obrázků

Obrázek 1: Ukázka návrhu uživatelského rozhraní v programu Figma.....	5
Obrázek 2: Schéma toku uživatele	6
Obrázek 3: Ukázka administračního rozhraní v programu Figma	7
Obrázek 4: Ukázka návrhu studentského rozhraní v programu Figma	7
Obrázek 5: Schéma databázové struktury	8
Obrázek 6: Implementovaná mapa s aktivními body.....	11
Obrázek 7: Výsledky Google Lighthouse	12
Obrázek 8: Formulář pro vytváření stanovišť	13
Obrázek 9: Schéma toku dat v aplikaci	14

Použité zdroje

1. **React.** <form>. *Learn React*. [Online] Meta Open Source, 2025. [Citace: 12. 04 2026.] <https://react.dev/reference/react-dom/components/form>.
2. —. React Reference Overview. *Learn React*. [Online] Meta Open Source, 2025. [Citace: 28. 03 2026.] <https://react.dev/reference/react>.
3. —. useMemo. *Learn React*. [Online] Meta Open Source, 2025. [Citace: 29. 03 2026.] <https://react.dev/reference/react/useMemo>.
4. **OpenAI.** ChatGPT. [Online] OpenAI, 2026. [Citace: 16. 03 2026.] <https://chatgpt.com/>.
5. **Anthropic.** Claude Sonnet. [Online] Claude, 2026. [Citace: 01. 05 2026.] <https://claude.ai/>.
6. **Microsoft.** Overview of ASP.NET Core. *Microsoft Learn*. [Online] Microsoft, 2026. [Citace: 30. 03 2026.] <https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-10.0>.
7. —. .NET API browser. *Microsoft Learn*. [Online] Microsoft, 2026. [Citace: 16. 03 2026.] <https://learn.microsoft.com/en-us/dotnet/api/?view=aspnetcore-10.0>.
8. —. Entity Framework Core. *Microsoft Learn*. [Online] Microsoft, 2026. [Citace: 06. 04 2026.] <https://learn.microsoft.com/en-us/ef/core/>.
9. **Vite contributors.** Vite. [Online] VoidZero Inc., 2026. [Citace: 11. 05 2026.] <https://devdocs.io/vite/>.
10. **Google.** Typography. *Google Material Design Guidelines*. [Online] Google, 2026. [Citace: 25. 03 2026.] <https://m3.material.io/styles/typography/overview>.
11. **IxDF.** Mobile First. *IxDF.org*. [Online] IxDF, 2025. [Citace: 29. 05 2026.] <https://ixdf.org/literature/topics/mobile-first>.
12. **WCAG.** Understanding the Web Content Accessibility Guidelines. *WCAG 101*. [Online] WCAG, 2026. [Citace: 06. 03 2026.] https://www.wcag.com/resource/what-is-wcag/#The_Three_Levels_of_WCAG_Conformance_A_AA_and_AAA.
13. **Github.** React zoom pan pinch. *Storybook*. [Online] Github, 30. 04 2026. [Citace: 09. 05 2026.] <https://bettertyped.github.io/react-zoom-pan-pinch/?path=/docs/docs-props--docs>.

A. Seznam příložených souborů

V Github repozitáři, ve složce „Dokumentace“ se nacházejí následující soubory:

- **RP2025-Pražáková-Nella-P3A-Průvodce_do_kapsy.docx** – editovatelná verze dokumentace maturitní práce
- **RP2025-Pražáková-Nella-P3A-Průvodce_do_kapsy.pdf** – tisknutelná verze dokumentace ročníkové práce
- **RP2025-Pražáková-Nella-P3A-Použití_AI.docx** – dokumentace využitých nástrojů umělé inteligence a ukázka promptů
- **RP2025-Pražáková-Nella-P3A-Pomocný-skript.docx** – pomocné skripty použité mimo repozitář projektu